

基于sc7 1684x的yolact模型测试完整版

1.环境部署

1.1 sdk开发包

```
1 wget https://sophon-file.sophon.cn/sophon-prod-s3/drive/23/06/15/16/Release_230701-public.zip
2 unzip Release_230701-public.zip
```

1.2 安装libsophon

uname -m: 查看系统架构

```
1 cd Release_230701-public/libsophon_20230806_182258
2 sudo dpkg -i sophon-*.deb(会有提示 *arm*.deb的文件无法安装: 因为该系统不是arm的, 忽略
  uname -m)
3 source /etc/profile
```

1.3 安装sophon-mw

依赖libsophon

```
1 cd Release_230701-public/sophon-mw_20230807_032400
2 sudo dpkg -i sophon-mw-sophon-ffmpeg_0.7.0_amd64.deb sophon-mw-sophon-ffmpeg-
  dev_0.7.0_amd64.deb
3 sudo dpkg -i sophon-mw-sophon-opencv_0.7.0_amd64.deb sophon-mw-sophon-opencv-
  dev_0.7.0_amd64.deb
4 source /etc/profile
```

1.4 安装sophon-sail

编译可被Python3接口调用的Wheel文件

sail编译与安装

使用**典型编译方式一** (PCIE MODE)

使用默认安装路径,编译包含bmcv,sophon-ffmpeg,sophon-opencv的SAIL

```
1 使用默认安装路径,编译包含bmcv,sophon-ffmpeg,sophon-opencv的SAIL
2 下载sophon-sail源码,解压后进入其源码目录
3 cd Release_230701-public/sophon-sail_20230802_085400
4 cd sophon-sail/
5 mkdir build && cd build
6 cmake .. #如果没有cmake, 按照提示安装sudo apt install cmake
7 make pysail
```

打包生成python wheel,生成的wheel包的路径为'python/pcie/dist',文件名为'sophon-3.6.0-py3-none-any.whl'

```
1 | cd ../python/pcie
2 | chmod +x sophon_pcie_whl.sh
3 | ./sophon_pcie_whl.sh
```

安装python wheel

```
1 | pip install ./dist/sophon-3.6.0-py3-none-any.whl --force-reinstall
2 | #pip install --upgrade pip
```

1.5 交叉编译环境搭建

安装工具链

```
1 | sudo apt-get install gcc-aarch64-linux-gnu g++-aarch64-linux-gnu
```

sophon-img

解压sophon-img包里的libsophon_soc<x.y.z>aarch64.tar.gz，将lib和include的所有内容拷贝到soc-sdk文件夹

```
1 | cd sophon-img_20230810_221353
2 | # 创建依赖文件的根目录
3 | mkdir -p soc-sdk
4 | # 解压sophon-img release包里的libsophon_soc_0.4.9_aarch64.tar.gz
5 | tar -zxvf libsophon_soc_0.4.9_aarch64.tar.gz
6 | # 将相关的库目录和头文件目录拷贝到依赖文件根目录下
7 | cp -rf libsophon_soc_0.4.9_aarch64/opt/sophon/libsophon-0.4.9/lib ${soc-sdk}
8 | cp -rf libsophon_soc_0.4.9_aarch64/opt/sophon/libsophon-0.4.9/include ${soc-sdk}
```

sophon-mw

```
1 | cd sophon-mw_20230807_032400
2 | # 解压sophon-mw包里的sophon-mw-soc_0.7.0_aarch64.tar.gz，其中x.y.z为版本号
3 | tar -zxvf sophon-mw-soc_0.7.0_aarch64.tar.gz
4 | # 将ffmpeg和opencv的库目录和头文件目录拷贝到依赖文件根目录下
5 | cp -rf sophon-mw-soc_0.7.0_aarch64/opt/sophon/sophon-ffmpeg_0.7.0/lib ${soc-sdk}
6 | cp -rf sophon-mw-soc_0.7.0_aarch64/opt/sophon/sophon-ffmpeg_0.7.0/include ${soc-sdk}
7 | cp -rf sophon-mw-soc_0.7.0_aarch64/opt/sophon/sophon-opencv_0.7.0/lib ${soc-sdk}
8 | cp -rf sophon-mw-soc_0.7.0_aarch64/opt/sophon/sophon-opencv_0.7.0/include ${soc-sdk}
```

验证

```
1 | which aarch64-linux-gnu-g++
2 | # 终端输出内容
3 | # /usr/bin/aarch64-linux-gnu-g++
```

1.6 tpu-mlir

主要量化工具

安装docker

```
1 | 以下为首次安装时需要的配置:
2 | # 安装docker
3 | sudo apt-get install docker.io
4 | # docker命令免root权限执行
5 | # 创建docker用户组, 若已有docker组会报错, 没关系可忽略
6 | sudo groupadd docker
7 | # 将当前用户加入docker组
8 | sudo gpasswd -a ${USER} docker
9 | # 重启docker服务
10 | sudo service docker restart
11 | # 切换当前会话到新group或重新登录重启X会话
12 | newgrp docker
13 |
14 | #添加镜像
15 | docker pull sophgo/tpuc_dev:v2.2
16 | docker pull sophgo/tpuc_dev:latest
```

2.部署 yolact

使用tpu-mlir

2.1. 前期准备

创建docker

注意TPU-MLIR工程在docker中的路径应该是/workspace/tpu-mlir

```
1 | 在Release_230701-public/tpu-mlir_20230802_054500下运行docker命令
2 |
3 | docker run --privileged --name name -v $PWD:/workspace -it
   | sophgo/tpuc_dev:latest
```

配置路径

```
1 | tar -zxf tpu-mlir_v1.2.8-g32d7b3ec-20230802.tar.gz
2 | source tpu-mlir_v1.2.8-g32d7b3ec-20230802/envsetup.sh
3 | 可以输入model_transform.py测试环境
4 |
```

下载算法

```
1 git clone https://github.com/sophon-ai-algo/examples.git
2 # Yolact示例项目代码位置 /examples/simple/yolact
3
4 # 通过脚本下载需要的数据和模型
5 cd examples/simple/yolact/
6 # 执行脚本下载数据和模型
7 ./scripts/download.sh
8
9 注：如果遇到断网，即timeout，可以在github上面搜索yolact_base_54_800000.pth手动下载
```

拷贝数据集

```
1 cp -rf $TPUC_ROOT/regression/image .
2 cp -rf $TPUC_ROOT/regression/dataset/COCO2017/ .
3 mkdir workspace && cd workspace
```

2.2 模型转换

pytorch模型转onnx模型

```
1 #使用示例项目代码中自带的模型转换脚本，可以将pytorch模型转为onnx模型：
2 python3 ../scripts/converter/convert.py --input
   ../data/models/yolact_base_54_800000.pth --mode onnx --cfg yolact_base
3
4 #移动onnx模型到当前目录
5 mv ../scripts/converter/yolact_base_54_800000.onnx .
```

ONNX转MLIR

```
1 # 创建模型转换命令脚本并执行
2 vi onnx2mlir.sh
3 sh onnx2mlir.sh
4
5 # onnx2mlir.sh中的内容
6 model_transform.py \
7     --model_name yolact \
8     --model_def ./yolact_base_54_800000.onnx \
9     --input_shapes [[1,3,550,550]] \
10    --mean 0.0,0.0,0.0 \
11    --scale 0.0039216,0.0039216,0.0039216 \
12    --keep_aspect_ratio \
13    --pixel_format rgb \
14    --test_input ../COCO2017/0000000000632.jpg \
15    --test_result yolact_top_outputs.npz \
16    --mlir yolact.mlir
```

转成mlir文件后，会生成一个 `${model_name}_in_f32.npz` 文件，该文件是模型的输入文件。

MLIR转F32模型

```
1 # 创建模型转换命令脚本并执行
2 vi mlir2bmodel_f32.sh
3 sh mlir2bmodel_f32.sh
4
5 # mlir2bmodel_f32.sh中的内容
6 model_deploy.py \
7     --mlir yolact.mlir \
8     --quantize F32 \
9     --chip bm1684x \
10    --tolerance 0.99,0.99 \
11    --test_input yolact_in_f32.npz \
12    --test_reference yolact_top_outputs.npz \
13    --model yolact_1684x_f32.bmodel
```

编译完成后, 会生成名为 `${model_name}_1684x_f32.bmodel` 的文件。

MLIR转INT8模型

生成校准表

转INT8模型前需要跑calibration, 得到校准表, 然后用校准表, 生成对称或非对称bmodel; 如果对称符合需求, 一般不建议用非对称, 因为 非对称的性能会略差于对称模型。

用现有的100张来自COCO2017的图片举例, 执行calibration, 这一步可能等待的时间较长:

```
1 vi run_cali.sh
2 sh run_cali.sh
3
4 #run_cali.sh的内容如下
5 run_calibration.py yolact.mlir \
6     --dataset ../COCO2017 \
7     --input_num 100 \
8     -o yolact_cali_table
```

运行完成后会生成名为 `${model_name}_cali_table` 的文件, 该文件用于后续编译INT8 模型的输入文件。

编译为INT8对称量化模型

```
1 # 创建模型转换命令脚本并执行
2 vi mlir2bmodel_int8_sym.sh
3 sh mlir2bmodel_int8_sym.sh
4
5 # mlir2bmodel_int8_sym.sh内容如下
6 model_deploy.py \
7     --mlir yolact.mlir \
8     --quantize INT8 \
9     --calibration_table yolact_cali_table \
10    --chip bm1684x \
11    --test_input yolact_in_f32.npz \
12    --test_reference yolact_top_outputs.npz \
13    --tolerance 0.85,0.45 \
14    --model yolact_1684x_int8_sym.bmodel
```

编译完成后, 会生成名为 `${model_name}_1684x_int8_sym.bmodel` 的文件。

编译为INT8非对称量化模型

```
1 # 创建模型转换命令脚本并执行
2 vi mlir2bmodel_int8_asym.sh
3 sh mlir2bmodel_int8_asym.sh
4
5 # mlir2bmodel_int8_asym.sh内容如下
6 model_deploy.py \
7     --mlir yolact.mlir \
8     --quantize INT8 \
9     --asymmetric \
10    --calibration_table yolact_cali_table \
11    --chip bm1684x \
12    --test_input yolact_in_f32.npz \
13    --test_reference yolact_top_outputs.npz \
14    --tolerance 0.90,0.55 \
15    --model yolact_1684x_int8_asym.bmodel
```

编译完成后, 会生成名为 `${model_name}_1684x_int8_asym.bmodel` 的文件。

2.3 Yolact模型推理测试

准备示例程序

将Yolact_1684x_f32.bmodel、Yolact_1684x_int8_asym.bmodel、Yolact_1684x_int8_sym.bmodel三个文件移动到/data/models下

```
1 mv Yolact_1684x_f32.bmodel Yolact_1684x_int8_asym.bmodel
   Yolact_1684x_int8_sym.bmodel ../data/models
2 # 退回至yolact/python目录
3 cd ../python
```

```
1 export LD_LIBRARY_PATH=/opt/sophon/sophon-opencv-
   latest/lib/:$LD_LIBRARY_PATH
2 export LD_LIBRARY_PATH=/opt/sophon/sophon-ffmpeg-
   latest/lib/:$LD_LIBRARY_PATH
3 export LD_LIBRARY_PATH=/opt/sophon/sophon-bmcpu-
   latest/lib/:$LD_LIBRARY_PATH
4 export LD_LIBRARY_PATH=/opt/sophon/sophon-libsophon-
   current/lib/:$LD_LIBRARY_PATH
```

部署测试

```
1 # sail解码 + bmcv预处理 + sail推理 + opencv后处理
2 # 下面的model参数Yolact_1684x_int8_asym.bmodel、Yolact_1684x_int8_sym.bmodel
3 python3 yolact_bmcv.py --cfgfile configs/yolact_base.cfg --model
  ../data/models/Yolact_1684x_f32.bmodel --is_video 0 --input_path ../image/
4 # 执行完毕后，在当前目录生成result_bmcv文件夹，检测结果保存在该文件夹下。
5
6 # opencv解码 + opencv预处理 + sail推理 + opencv后处理
7 # 下面的model参数Yolact_1684x_int8_asym.bmodel、Yolact_1684x_int8_sym.bmodel
8 python3 yolact_sail.py --cfgfile configs/yolact_base.cfg --model
  ../data/models/Yolact_1684x_f32.bmodel --is_video 0 --input_path ../image/
9 # 执行完毕后，在当前目录生成result_cv文件夹，检测结果保存在该文件夹下。
```

```
1 python3 yolov5_opencv_3output.py --bmodel
  ../workspace/yolov5s_1684x_f32.bmodel --input ../image/dog.jpg
```

- 运行以下命令测试开发和编译环境搭建成功

```
#输出相关设备信息即为成功
bm-smi
#输出相关参数帮助信息即为成功
bmrt_test
```